

第5章 上下文无关文法2026

第5章 概述

- 介绍另一类语言，上下文无关语言（CFL），使用了我们熟悉的术语：句子、句型、语法、语法树等，这容易让人产生联想而不感到陌生。
- 在Chomsky体系中，上下文无关语言是比正则语言范围更大的一类语言。
- CFL有两种语言识别器，一种是上下文无关文法（CFG）类似于正则表达式但更强，另一种是堆栈自动机（PDA）类似于有限自动机但也更强。

- 对应于教材第五章，我们从CFG开始，介绍与CFG有关的定义、推导、语法树、二义性、CFG化简等概念。目的是为语法分析做必要准备。

知识图谱

- CFL: (识别器:(CFG PDA) 判定性质:(推导 归约 语法树 移动) 等价性质:(文法清理 CFG~PDA PDA P~PDA N)
- 语法分析:(原理:(LL(1) LR(0) SLR(1)) 框架:(SRF itemDFA itemDPDA) 程序实现:(递归下降 预测分析 SLR(1)表驱动 自动生成))

5.1 文法如何定义语言

- 上下文无关语言：
  - 回文 $L_{\text{pal}} = \{w \in \{0,1\}^* \mid w = w^{\text{R}}\}$
  - $L^{\text{DUP}} = \{ww \mid w \in L, L \subseteq \{0,1\}^*\}$
- 文法是生成语言的规则，以回文为例
  - 基础：ε、 0、1 都属于回文；
  - 归纳：若w属于回文，那么$0w0$和$1w1$都属于回文。
  - 文法G_{pal}
    - $A \longrightarrow \varepsilon \mid 0 \mid 1$
    - $A \longrightarrow 0A0 \mid 1A1$
  - $A \Rightarrow 0A0 \Rightarrow 01A10 \Rightarrow 010A010 \Rightarrow 0101010$
  - $L_{\text{pal}} = L(G_{\text{pal}}) = \{w \in \{0,1\}^* \mid A \Rightarrow_{G\text{pal}}{}^* w\}$

CFG允许多个变元

- 例：$\{xy \mid x,y \in L_{\text{pal}}\}$
  - $S \longrightarrow AA$
  - $A \longrightarrow \varepsilon \mid 0 \mid 1$
  - $A \longrightarrow 0A0 \mid 1A1$

- 变元指代的语言就是从它能推导出的终结符串的集合
- CFG中有一个变元是文法初始符号，它的语言也就是这个文法所定义的语言。
- CFG识别的语言是它的初始符号所指代的语言，这是上下文无关语言。

CFG的形式定义

- CFG $G=(V,T,\mathcal{P},S)$，其中，
  - G为文法标识（可省略），
  - V是变元的有限集，
  - T是终结符的有限集（字母表，也习惯用Σ），
  - $\mathcal{P}$是产生式规则的有限集，
  - S是文法初始符号，$S\in V$。
- 文法G所定义的语言，$L(G)=L(S)\subseteq T^*$，是T上的语言。

符号使用惯例

在形式语言中，变元集V、终结符集或字母表T、产生式或规则集P、初始符号S

在编译原理中，非终结符集V_N、终结符集V_T、产生式（规则）集$\mathcal{P}$、开始符号S、文法符号集 $V=V_N\cup V_T$

CFG形式定义举例

- CFG $G_{\text{pal}} = (V, T, \boldsymbol{\mathcal{P}}, S)$
  - $V = \{A\}$
  - $T = \{0,1\}$
  - $\boldsymbol{\mathcal{P}}$: $A \longrightarrow \varepsilon \mid 0 \mid 1$ $A \longrightarrow 0A0 \mid 1A1$
  - $S = A$
- CFG $G_{\text{pal}} = (\{A\}, \{0,1\}, \{(A, \varepsilon), (A, 0), (A, 1), (A, 0A0), (A, 1A1)\}, A)$
- $L(G_{\text{pal}}) = L_{\text{pal}} = \{w \in \{0, 1\}^* \mid w = w^{\text{R}}\}$
- $L(G_{\text{pal}}) = L(A) = \{w \in \{0, 1\}^* \mid A \Rightarrow_{G\text{pal}}{}^* w\}$

CFG形式定义（续）

- 对于任意CFG $G=(V,T,\mathcal{P},S)$，有，
- $V\cap T=\emptyset$若$(A,\alpha)\in\mathcal{P}$, 那么$A\in V$, 并且$\alpha\in(V\cup T)^*$
- 一般直观地写成：$A\longrightarrow\alpha$
- A是产生式左部，α是右部，为了方便α又称为A的候选式
- 如果A的全部候选式为$\alpha_1...\alpha_k$，$k>1$，那么可简写为，
  - $A\longrightarrow\alpha_1|...|\alpha_k$
- $S\in V$（恰有一个变元是文法初始符号）
- $L(G)=L(S)\subseteq T^*$

符号使用习惯：

A，B，C,… 为变元

a，b，c,… 为终结符

…，X，Y，Z 为终结符或者变元

符号使用习惯：

…，w，x，y，z 只是终结符串

α，β，γ,… 终结符和变元组成的串

第一个产生式的左部变元为文法初始符号

例：算术表达式

- $E \longrightarrow E + E \mid E * E \mid (E) \mid d$
- 变元集合 $\{E\}$
- 终结符集合 $\{+, *, (,), d\}$
- 产生式规则集合，4 个如上所示
- 文法初始符号 E

- CFG $(\{E\}, \{+, *, (,), d\}, \{(E, E+E), (E, E*E), (E, \backslash(E\backslash)), (E, d)\}, E)$

例 声明

- $\check{D} \longrightarrow \varepsilon \mid D; \check{D}$
- $D \longrightarrow T\ d$
- $T \longrightarrow \text{int} \mid \text{float}$
- 回顾词法记号与词法单元概念
  - d, int, float和;构成文法的终结符集合。
- 用到的语言？
  - $\check{D}, D, T$构成文法的变元集合，其中$\check{D}$为文法初始符号。
- 代数表示为CFG $(\{\check{D}, D, T\}, \{d, \text{int}, \text{float}, ;\},$
  $\{(\check{D}, \varepsilon), (\check{D}, D;\check{D}), (D, Td), (T, \text{int}), (T, \text{float})\}, \check{D})$
- $L(T)$=?； $L(D)$=?； $L(\check{D})$=?
  - RE：$((\text{int}\ d + \text{float}\ d);)^*$； 采用推导$\check{D} \Rightarrow \ldots$

推导

- 对于CFG $G=(V,T,\boldsymbol{P},S)$，
- 符号串 η 推导 δ 写为 $\eta \Rightarrow \delta$，其中 $\eta,\delta \in (V \cup T)^*$
- 基本思想是将产生式规则看作重写规则，如果串η中某个变元的一次出现被它的某个候选式替换，得到串δ，就说η推导δ
- 形式地，如果$\eta=\alpha A\beta$，$\delta=\alpha\gamma\beta$，$(A,\gamma)\in\boldsymbol{P}$，那么$\eta \Rightarrow \delta$

- 例：CFG $G=(\{A,B\},\{0,1\},\{(B,AA),(A,01),(A,0A1)\},B)$
- $00A11 \Rightarrow 000111$
- $00A11 \Rightarrow 000A111$
- $A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 00001111$
- $B \Rightarrow AA \Rightarrow 0A1A \Rightarrow 0011A \Rightarrow 00110A1 \Rightarrow 00110011$

推导

- 这种推导可以一直进行下去，串中任何变元的一次出现都可以用它的某个候选式来替换，直到串中只剩下终结符为止。
- $B \Rightarrow AA \Rightarrow 0A1A \Rightarrow 0011A \Rightarrow 00110A1 \Rightarrow 00110011$
- $E \Rightarrow E+E \Rightarrow E+E*E \Rightarrow E+d*E \Rightarrow E+d*d \Rightarrow d+d*d$
- $\check{D} \Rightarrow D;\check{D} \Rightarrow D;D;\check{D} \Rightarrow Td;D;\check{D} \Rightarrow Td;Td;\check{D} \Rightarrow Td;Td; \Rightarrow$ int d;Td; $\Rightarrow$ int d;float d;

- 如果从文法初始符号开始这个推导过程，终将得到该文法定义的语言的每一个成员。

推导与直接推导

- 对于CFG $G=(V,T,\boldsymbol{\mathcal{P}},S)$，
- 如果$\eta=\alpha A\beta$，$\delta=\alpha\gamma\beta$，$(A,\gamma)\in\boldsymbol{\mathcal{P}}$，$\alpha,\beta\in(V\cup T)^*$，那么$\eta\Rightarrow\delta$。
- $\Rightarrow$称为直接推导。

- 推导$\Rightarrow^*$是指“0到多个直接推导步骤”
- 基础：$\eta\Rightarrow^*\eta$，对任意$\eta\in(V\cup T)^*$
- 归纳：若 $\eta\Rightarrow^*\alpha$， $\alpha\Rightarrow\delta$，那么 $\eta\Rightarrow^*\delta$，对任意$\eta,\alpha,\delta\in(V\cup T)^*$

例子：推导与直接推导

- $S \longrightarrow 01;\ S \longrightarrow 0S1$。
  - $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000111$ 。
- 所以
  - $S \Rightarrow^{*} S$;
  - $S \Rightarrow^{*} 0S1$;
  - $S \Rightarrow^{*} 00S11$;
  - $S \Rightarrow^{*} 000111$。

- $E \Rightarrow E+E \Rightarrow E+E*E \Rightarrow E+d*E \Rightarrow E+d*d \Rightarrow d+d*d$
- $\check{D} \Rightarrow D;\check{D} \Rightarrow D;D;\check{D} \Rightarrow Td;D;\check{D} \Rightarrow Td;Td;\check{D} \Rightarrow Td;Td; \Rightarrow \text{int}\ d;Td;$ $\Rightarrow \text{int}\ d;\text{float}\ d;$

推导与直接推导的性质

- 推导$\Rightarrow^*$
  - 自反性
  - 传递性

- 直接推导$\Rightarrow$
  - 重写性质， 即，

对任意文法符号串α和β，$\alpha A\beta \Rightarrow \alpha\gamma\beta$当且仅当$(A,\gamma)\in\boldsymbol{\mathcal{P}}$

注意，替换$\alpha A\beta$中的A为γ得到$\alpha\gamma\beta$与α和β都无关。

归约与直接归约

- 从产生式的右部到左部来运用，一步一步地进行，并最终将给定的终结符号串转变为文法初始符号，或者因所有子串都不是候选式而结束。
- 在过程的每一步，当前符号串中的某个子串与某个候选式相同，则可以将这个子串用那个候选式的变元替代，从而当前符号串发生改变。
- 对于CFG $G=(V,T,\boldsymbol{P},S)$，如果$(A,\gamma)\in\boldsymbol{P}$，$\alpha,\beta\in(V\cup T)^*$，那么有$\alpha\gamma\beta\Leftarrow\alpha A\beta$。$\Leftarrow$称为直接归约，是直接推导的逆过程。
- 连续归约$\Leftarrow^*$是指“0到多个直接归约步骤”，定义为，
  - 基础：$\eta\Leftarrow^*\eta$，对任意$\eta\in(V\cup T)^*$
  - 归纳：若 $\eta\Leftarrow^*\alpha$，$\alpha\Leftarrow\delta$，那么 $\eta\Leftarrow^*\delta$，对任意$\eta,\alpha,\delta\in(V\cup T)^*$

例子：归约与直接归约

- $S \longrightarrow 01$ $S \longrightarrow 0S1$。
  - $000111 \Leftarrow 00S11 \Leftarrow 0S1 \Leftarrow S$。
- 所以
  - $S \Leftarrow^* S$;
  - $0S1 \Leftarrow^* S$;
  - $00S11 \Leftarrow^* S$;
  - $000111 \Leftarrow^* S$ 。

推导、归约与语言

- 除了自反性，推导关系的传递性还包括以下内容：
  - $\alpha \Rightarrow^* \gamma$如果有$\beta$使得$\alpha \Rightarrow \beta$且$\beta \Rightarrow \gamma$
  - $\alpha \Rightarrow^* \gamma$如果有$\beta$使得$\alpha \Rightarrow^* \beta$且$\beta \Rightarrow \gamma$
  - $\alpha \Rightarrow^* \gamma$如果有$\beta$使得$\alpha \Rightarrow^* \beta$且$\beta \Rightarrow^* \gamma$
- 借助于推导关系的语言定义。
  - 定义：给定CFG $G=(V, T, \mathcal{P}, S)$，
    该文法定义的语言$L(G) = \{w \in T^* \mid S \Rightarrow^* w\}$。

- 借助于归约关系的语言定义。
  - 定义：给定CFG $G=(V, T, \mathcal{P}, S)$，
    该文法定义的语言$L(G) = \{w \in T^* \mid w \Leftarrow^* S\}$。

例：文法生成符号串

- 示例：文法通过推导来生成符号串*aabbbcc*
- 替换策略：随机选取任意变元的任意一次出现替换之（推导）；固定选择最左（右）边的变元用它的某个候选式替换之（最左、最右推导）

$$
\begin{aligned}
T &\to R \\
T &\to \mathrm{a}T\mathrm{c} \\
R &\to \varepsilon \\
R &\to R\mathrm{b}R
\end{aligned}
$$

$$
\begin{aligned}
&\underline{T} \\
\Rightarrow\ & \mathrm{a}\underline{T}\mathrm{c} \\
\Rightarrow\ & \mathrm{aa}\underline{T}\mathrm{cc} \\
\Rightarrow\ & \mathrm{aa}\underline{R}\mathrm{cc} \\
\Rightarrow\ & \mathrm{aa}R\mathrm{b}\underline{R}\mathrm{cc} \\
\Rightarrow\ & \mathrm{aa}\underline{R}\mathrm{bcc} \\
\Rightarrow\ & \mathrm{aa}R\mathrm{b}\underline{R}\mathrm{bcc} \\
\Rightarrow\ & \mathrm{aa}R\mathrm{b}\underline{R}\mathrm{b}R\mathrm{bcc} \\
\Rightarrow\ & \mathrm{aa}\underline{R}\mathrm{bb}R\mathrm{bcc} \\
\Rightarrow\ & \mathrm{aabb}\underline{R}\mathrm{bcc} \\
\Rightarrow\ & \mathrm{aabbbcc}
\end{aligned}
$$

$$
\begin{aligned}
&\underline{T} \\
\Rightarrow\ & \mathrm{a}\underline{T}\mathrm{c} \\
\Rightarrow\ & \mathrm{aa}\underline{T}\mathrm{cc} \\
\Rightarrow\ & \mathrm{aa}\underline{R}\mathrm{cc} \\
\Rightarrow\ & \mathrm{aa}\underline{R}\mathrm{b}R\mathrm{cc} \\
\Rightarrow\ & \mathrm{aa}\underline{R}\mathrm{b}R\mathrm{b}R\mathrm{cc} \\
\Rightarrow\ & \mathrm{aab}\underline{R}\mathrm{b}R\mathrm{cc} \\
\Rightarrow\ & \mathrm{aab}\underline{R}\mathrm{b}R\mathrm{b}R\mathrm{cc} \\
\Rightarrow\ & \mathrm{aabb}\underline{R}\mathrm{b}R\mathrm{cc} \\
\Rightarrow\ & \mathrm{aabbb}\underline{R}\mathrm{cc} \\
\Rightarrow\ & \mathrm{aabbbcc}
\end{aligned}
$$

句型与句子

- 这里借用了自然语言的术语。
- **句型**：从文法初始符号推导出来的任意串都是句型。
- **句子**：从文法初始符号推导出来的终结符串都是句子。
- 句子是句型的一个特例。
- 形式地：
  - 对于文法$G=(V, T, \mathcal{P}, S)$，
  - α是一个句型，当且仅当 $S \Rightarrow^* \alpha$
  - w是一个句子，当且仅当 $S \Rightarrow^* w$且$w \in T^*$
- 上页例子中每一步推导得到的都是句型，只有最后一步是句子。
- 思考：CFG的语言是所有句子的集合。

最左（右）推导的符号表示

- 推导允许我们替换串中的任意变元，并允许使用它的任意候选式。
- 这种重写策略会导致同一个句型有多个推导存在。
- 最左（右）推导确定化了推导过程采用的重写策略。

- 最左推导：$\Rightarrow_{lm}$、$\overset{*}{\Rightarrow}_{lm}$
- 最右推导：$\Rightarrow_{rm}$、$\overset{*}{\Rightarrow}_{rm}$

例：平衡括号文法

- 平衡括号文法：

$$S \longrightarrow SS \mid (S) \mid ()$$

- $S \Rightarrow_{lm} SS \Rightarrow_{lm} (S)S \Rightarrow_{lm} (())S \Rightarrow_{lm} (())()$
- 因此，$S \Rightarrow^*_{lm} (())()$
- $S \Rightarrow SS \Rightarrow S() \Rightarrow (S)() \Rightarrow (())()$ 是一个推导，但不是最左推导

- $S \Rightarrow_{rm} SS \Rightarrow_{rm} S() \Rightarrow_{rm} (S)() \Rightarrow_{rm} (())()$
- 因此, $S \Rightarrow^*_{rm} (())()$
- $S \Rightarrow SS \Rightarrow SSS \Rightarrow S()S \Rightarrow ()()S \Rightarrow ()()()$ 既不是最左推导也不是最右推导

最左、最右推导

$E \longrightarrow E+E \mid E*E \mid (E) \mid F$

$F \longrightarrow aF \mid bF \mid 0F \mid 1F \mid \varepsilon$

问句子 a*(ab+10)的最左、最右推导？

$$
\begin{aligned}
&E \\
&\Rightarrow_{lm} E*E \\
&\Rightarrow_{lm} F*E \\
&\Rightarrow_{lm} a*E \\
&\Rightarrow_{lm} a*(E) \\
&\Rightarrow_{lm} a*(E+E) \\
&\Rightarrow_{lm} a*(F+E) \\
&\Rightarrow_{lm} a*(aF+E) \\
&\Rightarrow_{lm} a*(abF+E) \\
&\Rightarrow_{lm} a*(ab+E) \\
&\Rightarrow_{lm} a*(ab+F) \\
&\Rightarrow_{lm} a*(ab+1F) \\
&\Rightarrow_{lm} a*(ab+10F) \\
&\Rightarrow_{lm} a*(ab+10)
\end{aligned}
$$

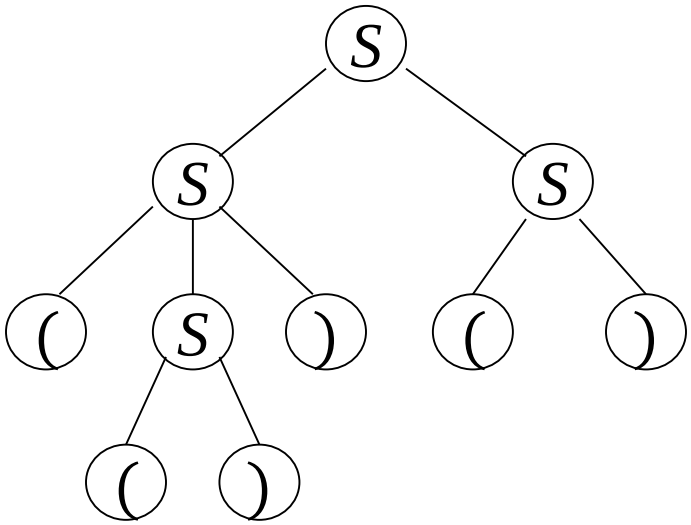
$$
\begin{aligned}
&E \\
&\Rightarrow_{rm} E*E \\
&\Rightarrow_{rm} E*(E) \\
&\Rightarrow_{rm} E*(E+E) \\
&\Rightarrow_{rm} E*(E+F) \\
&\Rightarrow_{rm} E*(E+1F) \\
&\Rightarrow_{rm} E*(E+10F) \\
&\Rightarrow_{rm} E*(E+10) \\
&\Rightarrow_{rm} E*(F+10) \\
&\Rightarrow_{rm} E*(aF+10) \\
&\Rightarrow_{rm} E*(abF+0) \\
&\Rightarrow_{rm} E*(ab+10) \\
&\Rightarrow_{rm} F*(ab+10) \\
&\Rightarrow_{rm} aF*(ab+10) \\
&\Rightarrow_{rm} a*(ab+10)
\end{aligned}
$$

5.2 语法分析树

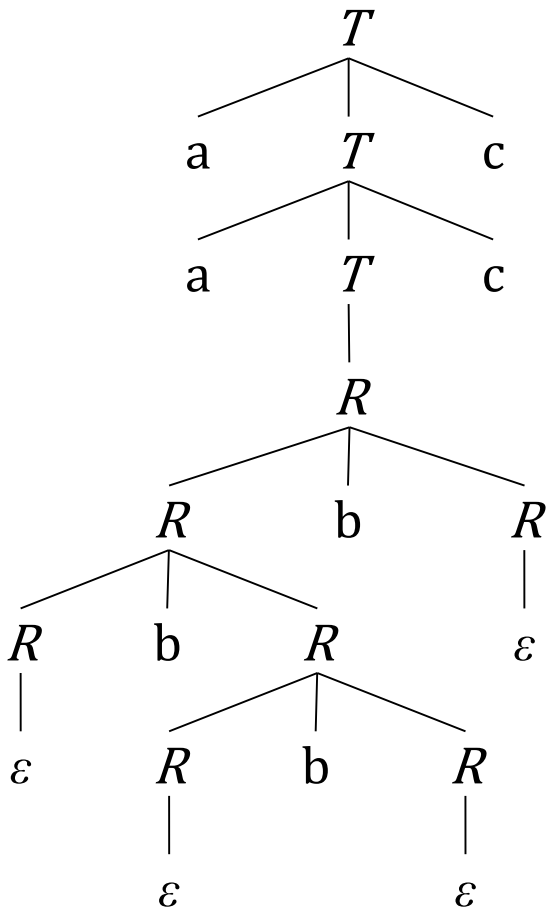
- 可以将推导写成树形表示。树根是文法初始符号。每一次直接推导所替换的变元，就是在树中给它加上孩子节点，即对于所用候选式，依序一个符号一个节点，这些节点都作为孩子。

  对于文法：$S \rightarrow SS \mid (S) \mid ()$，

  推导$S \Rightarrow SS \Rightarrow (S)S \Rightarrow (())S \Rightarrow (())()$的语法树



语法树示例



$$T \to R$$
$$T \to \mathrm{a}T\mathrm{c}$$
$$R \to \varepsilon$$
$$R \to R\mathrm{b}R$$

$$\begin{aligned}
&\underline{T}\\
\Rightarrow\ &\mathrm{a}\underline{T}\mathrm{c}\\
\Rightarrow\ &\mathrm{aa}\underline{T}\mathrm{cc}\\
\Rightarrow\ &\mathrm{aa}\underline{R}\mathrm{cc}\\
\Rightarrow\ &\mathrm{aa}R\mathrm{b}\underline{R}\mathrm{cc}\\
\Rightarrow\ &\mathrm{aa}\underline{R}\mathrm{bcc}\\
\Rightarrow\ &\mathrm{aa}R\mathrm{b}\underline{R}\mathrm{bcc}\\
\Rightarrow\ &\mathrm{aa}R\mathrm{b}\underline{R}\mathrm{b}R\mathrm{bcc}\\
\Rightarrow\ &\mathrm{aa}\underline{R}\mathrm{bb}R\mathrm{bcc}\\
\Rightarrow\ &\mathrm{aabb}\underline{R}\mathrm{bcc}\\
\Rightarrow\ &\mathrm{aabbbcc}
\end{aligned}$$

$$\begin{aligned}
&\underline{T}\\
\Rightarrow\ &\mathrm{a}\underline{T}\mathrm{c}\\
\Rightarrow\ &\mathrm{aa}\underline{T}\mathrm{cc}\\
\Rightarrow\ &\mathrm{aa}\underline{R}\mathrm{cc}\\
\Rightarrow\ &\mathrm{aa}\underline{R}\mathrm{b}R\mathrm{cc}\\
\Rightarrow\ &\mathrm{aa}\underline{R}\mathrm{b}R\mathrm{b}R\mathrm{cc}\\
\Rightarrow\ &\mathrm{aab}\underline{R}\mathrm{b}R\mathrm{cc}\\
\Rightarrow\ &\mathrm{aab}\underline{R}\mathrm{b}R\mathrm{b}R\mathrm{cc}\\
\Rightarrow\ &\mathrm{aabb}\underline{R}\mathrm{b}R\mathrm{cc}\\
\Rightarrow\ &\mathrm{aabbb}\underline{R}\mathrm{cc}\\
\Rightarrow\ &\mathrm{aabbbcc}
\end{aligned}$$

语法分析树

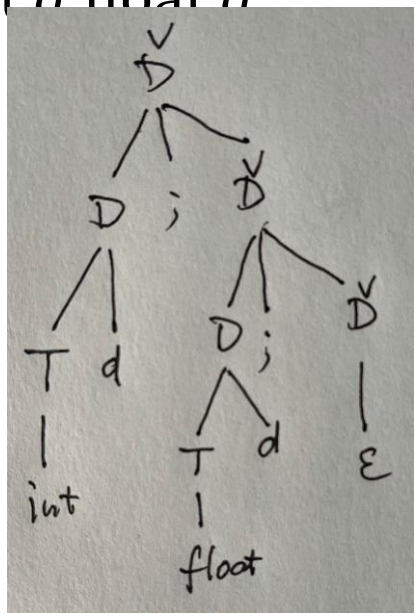
- 语法树的叶子是终结符或ε，将语法树的叶子从左往右依序连接成串，该串是句子。

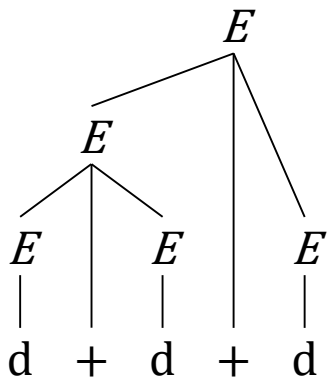
- 结论：CFL的每个句子都有自己的语法树，都是有序树。
  - 叶子：用终结符或ε标记
  - 内节点：用变元标记
  - 内节点的孩子对应变元的候选式（从左往右组成串）
  - 根：用文法初始符号标记

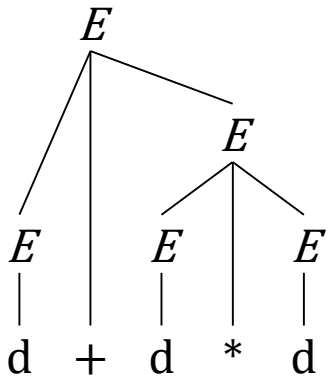
- 若允许语法树的叶子还可以是变元的话，我们就能得到句型的语法树。句型的语法树的叶子是变元、终结符或ε。

语法树的产物

- 一个句子的语法树的产物就是该句子（每个句子都有它的语法树）。
- 一个句型的语法树的产物就是该句型。
- 语法树给它的产物增加了结构信息，将为编译所使用。
- $\check{D} \Rightarrow D;\check{D} \Rightarrow D;D;\check{D} \Rightarrow Td;D;\check{D} \Rightarrow Td;Td;\check{D} \Rightarrow Td;Td; \Rightarrow$ int d;Td; $\Rightarrow$ int d;float d;







与CFG判定性质有关的等价性

- CFG $G=(V, T, P, S)$的判定过程的等价性。

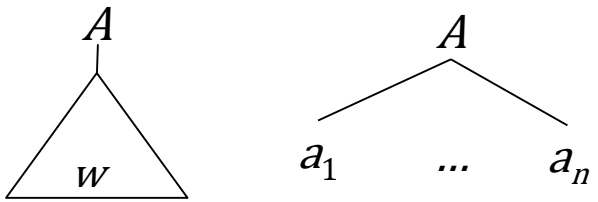
- 结论：对于任意$A\in V$，$w\in T^*$，以下彼此是等价的。

1. （归约）$w \Leftarrow^* A$ 根据归约知道$w\in L(A)$。
2. （推导）$A \Rightarrow^* w$ 根据推导知道$w\in L(A)$。
3. （最左推导）$A \Rightarrow^*_{\mathrm{lm}} w$ 根据最左推导知道$w\in L(A)$。
4. （最右推导）$A \Rightarrow^*_{\mathrm{rm}} w$ 根据最右推导知道$w\in L(A)$。
5. （语法树）存在根节点为A，产物为w的语法树。

从判定性1（归约）到5（语法树）

- **定理**5.1 设文法$G=(V,T,P,S)$是一个CFG，对任意终结符串w，如果w归约为A，则一定存在一颗以A为根、产物为w的语法树。
- 证明，对归约步数进行归纳。
- 基础：若该归约步数只有一步，那么是直接归约，即存在产生式规则$A\longrightarrow w$，$w\Leftarrow^{*}A$。那么有一颗以w个符号标记的叶子，以A标记的根，组成树，即是w的语法树，产物为w。

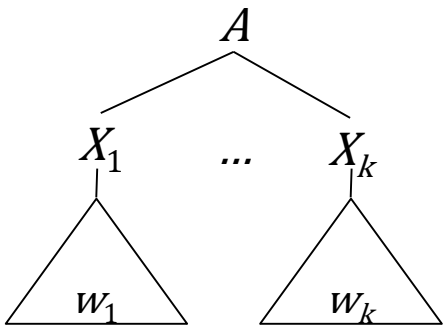
$A\longrightarrow w$，$w\Leftarrow^{*}A$



从归约到语法树

- 归纳：假设对于步数小于n的归约$w \Leftarrow^* A$，都存在以A为根以w为产物的语法树。那么归约步数为n时，观察最后一步归约，即归约为A，一定使用了A的产生式，不失一般性假设为$A \longrightarrow X_1 ... X_k$，其中$X_i \in V$或者$X_i \in T$，对于$i=1,...,k$。那么$w$一定可以分成$k$段，即$w=w_1 ... w_k$，并且$w_i \Leftarrow^* X_i$，对于$i=1,...,k$。因此存在根为$A$，$A$的孩子为$X_1 ... X_k$，产物为$w$的语法树。

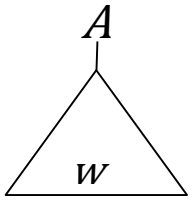
$$A \longrightarrow X_1 ... X_k$$
$$X_1 ... X_k \Leftarrow^* A$$
$$w = w_1 ... w_k$$
$$w_i \Leftarrow^* X_i, \quad i=1,...,k$$



从判定性5（语法树）到3（推导）

- 定理5.2。设文法$G=(V,T,\mathcal{P},S)$是一个CFG，若有一个语法树它的根标记为变元A并且产物是w，$w\in T^*$，那么一定存在一个G中的最左推导$A\Rightarrow^*_{\text{lm}} w$。
- 证明，对语法树的高度进行归纳。
- 基础：树高为1，那么树的叶子只能是终结符，树根为A。那么树的叶子从左往右连接起来为w。由于这是一个语法树，所以$A\longrightarrow w$一定是$\mathcal{P}$中产生式，因此有最左推导$A\Rightarrow^*_{\text{lm}} w$。

树高为1

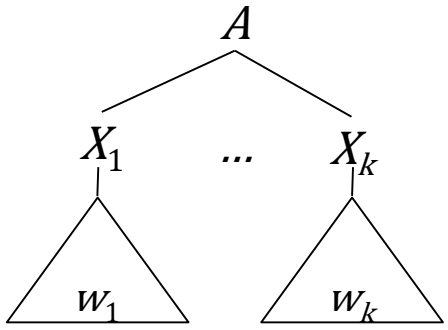


有$A\longrightarrow w$，
$A\Rightarrow^*_{\text{lm}} w$

从语法树到推导

- 归纳：假设对于高度小于n的语法树，其中根为X产物为x，都存在$X \Rightarrow^*_{\text{lm}} x$。那么，当树高为$n$时，我们总可以把它分解为树根及其孩子，以及每个以孩子为根的子树。不失一般性，假设树根为A，孩子为$X_1 \ldots X_k$，由于这是语法树，所以必定有产生式$A \longrightarrow X_1 \ldots X_k$，故$A \Rightarrow^*_{\text{lm}} X_1 \ldots X_k$。另每个以$A$的孩子为根的子树的高度都小于$n$，根据归纳假设，它们对应有最左推导$X_i \Rightarrow^* w_i$，对于$i=1,\ldots,k$。其中$w=w_1 \ldots w_k$。
- 因此，$A \Rightarrow_{\text{lm}} X_1 \ldots X_k \Rightarrow^*_{\text{lm}} w_1 X_2 \ldots X_k \Rightarrow^*_{\text{lm}} w_1 w_2 X_3 \ldots X_k \Rightarrow^*_{\text{lm}} \cdots \Rightarrow^*_{\text{lm}} w_1 \ldots w_k$

子树高≥ 0



$A \longrightarrow X_1 \ldots X_k$

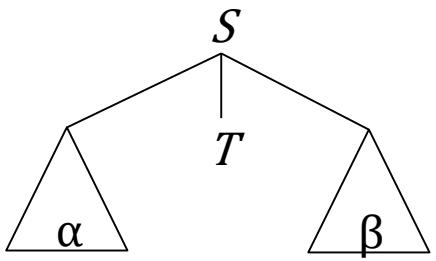
$A \Rightarrow^* X_1 \ldots X_k \Rightarrow^* w = w_1 \ldots w_k$

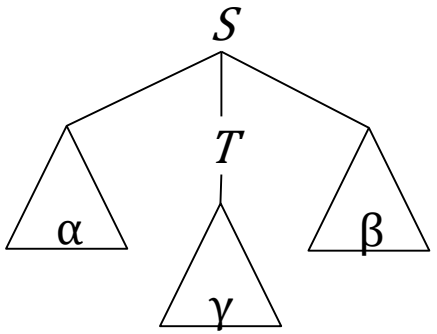
$X_i \Rightarrow^* w_i$，$i=1,\ldots,k$

推导步骤与扩展语法树步骤一一对应

- 在过程步骤上，推导过程与扩展语法树过程一一对应。
- 从根S扩展产物为w的语法树的过程，初始时S为可扩展节点，
  - 对于当前可扩展节点，用它的候选式作为孩子节点；
  - 若当前节点为未扩展过的变元则它为可扩展节点；
  - 扩展语法树直到没有可扩展节点时结束。
  - 当语法树与推导策略无关时，从给定的推导得到与语法树扩展的对应关系

$$\alpha T\beta \Rightarrow \alpha\gamma\beta \text{如果有产生式} T \longrightarrow \gamma$$





归约步骤与构建语法树步骤一一对应

- w 归约为S，对应语法树根S和产物w
- 符号串$w \in T^*$的归约是一个序列$\alpha_1,\ldots,\alpha_n, n>1$。其中$\alpha_1 = w$，$\alpha_n = S$，$\alpha_{i+1} \Rightarrow \alpha_i, 1 \leqslant i < n$。记为$w \Leftarrow^* \alpha_n$。

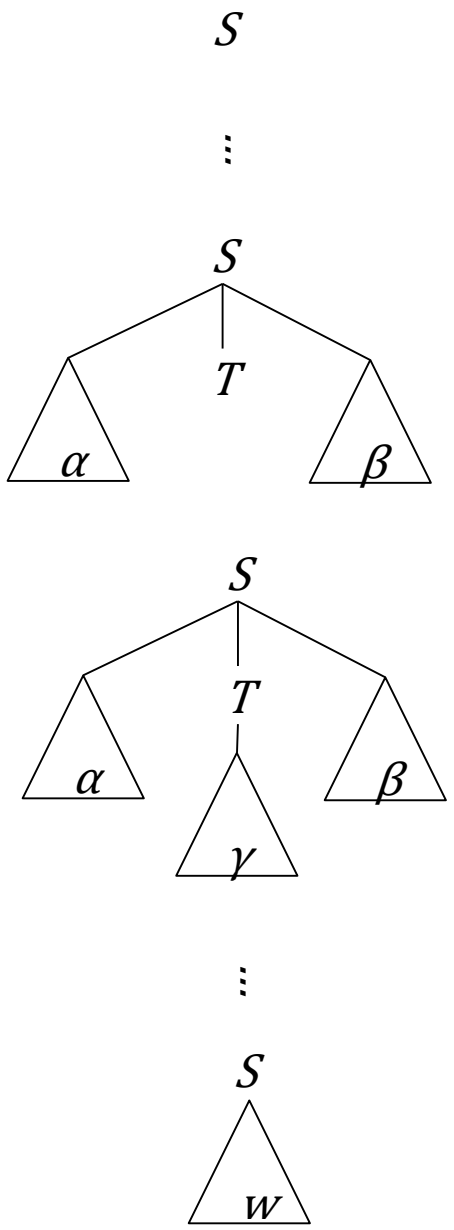
- 语法树的生成过程：
- 从根向叶子方向扩展语法树：对应推导。
- 从叶子往根方向构建语法树：对应归约。

推导、归约与语法树对应关系

$S \Rightarrow^* w$

$S \Rightarrow^* \alpha T \beta$
$\Rightarrow \alpha\gamma\beta$
$\Rightarrow^* w$

自上而下地扩展



$w \Leftarrow^* S$

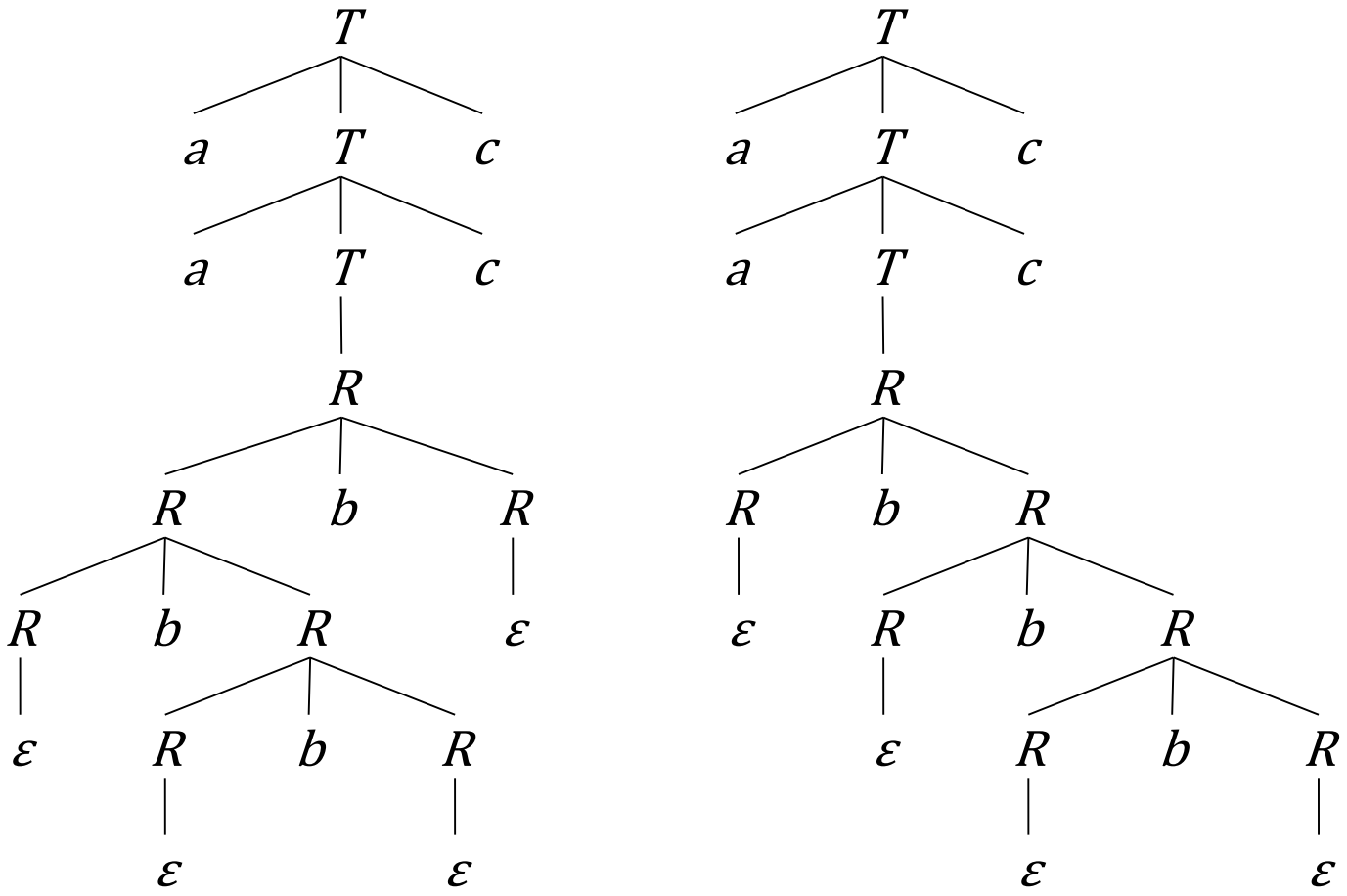
$w \Leftarrow^* \alpha\gamma\beta$
$\Leftarrow \alpha T \beta$
$\Leftarrow^* S$

自下而上地构建

5.4 二义性文法

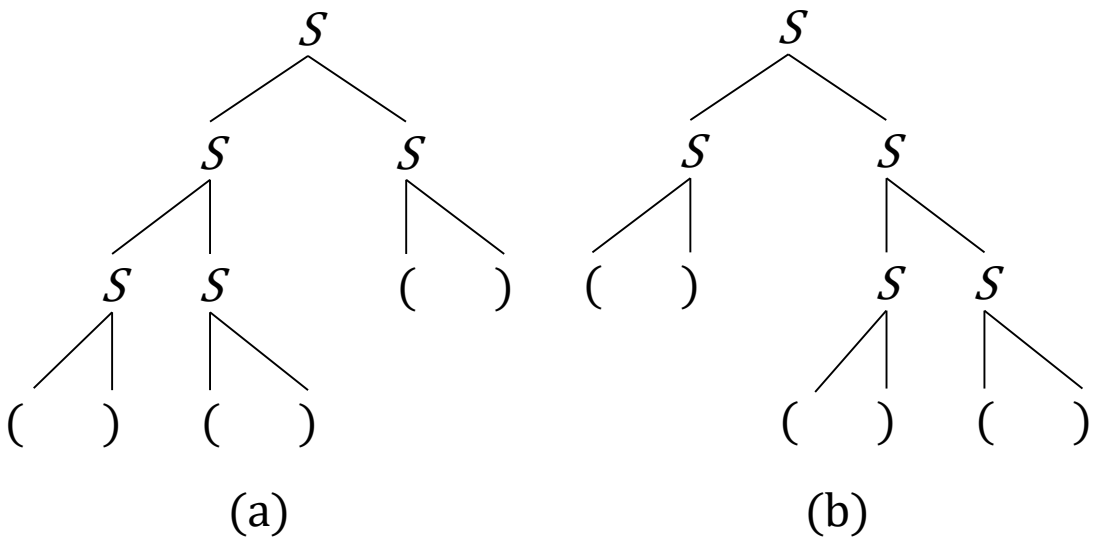
- 选择不同的候选式常常会推导出不同的句子，但有时不尽然。如果真出现同一个句子有多个语法树的情况，表明文法是有二义的。

$T \to R$
$T \to \mathrm{a}T\mathrm{c}$
$R \to \varepsilon$
$R \to R\mathrm{b}R$



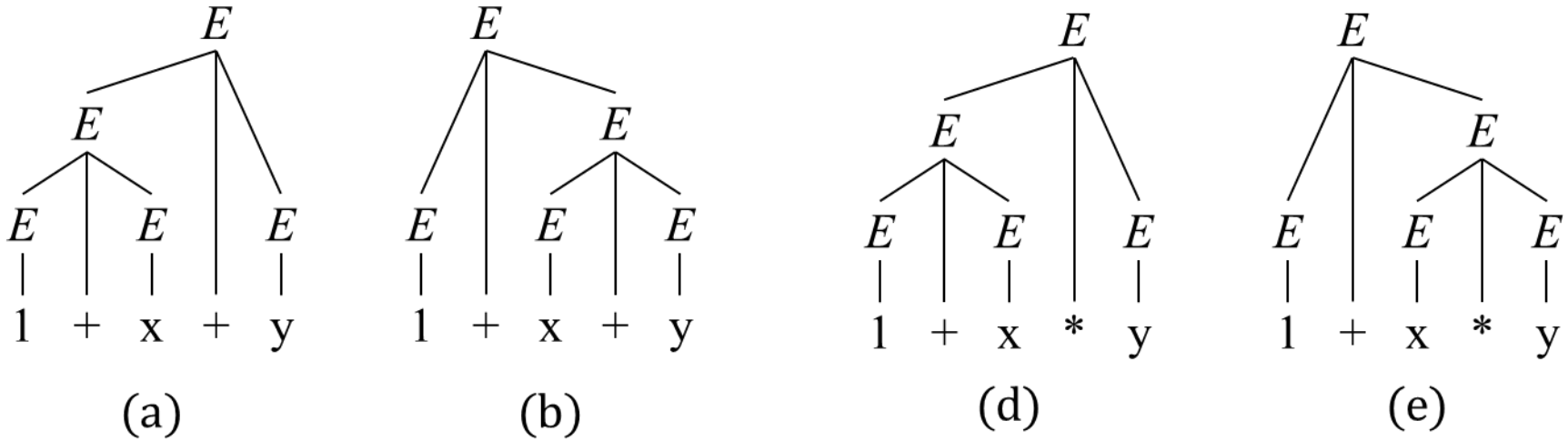
二义性文法

- 一个 CFG 是二义的 若它的语言有元素至少是两个语法树的产物。
- 例：平衡括号语言 $S \rightarrow SS \mid (S) \mid ()$
- 容易找到句子它有两个语法树



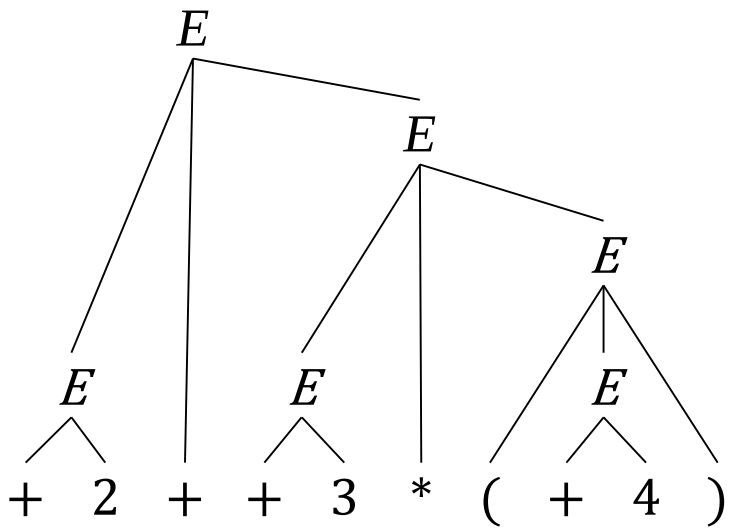
二义性文法

- $E \longrightarrow E+E \mid E*E \mid (E) \mid d \mid i$

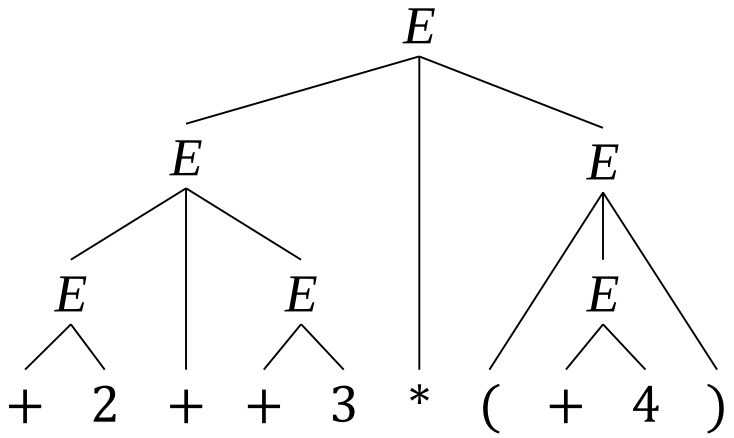


二义性文法

- $E \longrightarrow E + E \mid E * E \mid + i \mid (E) \mid d \mid i$



(a)



(b)

二义性与最左/最右推导

- 定理5.5 对于CFG的句子w有两个语法树当且仅当有两个最左推导。
- 如果w有两个语法树，根据定理证明中的构造方法必定存在两个不同的最左推导。
- 相反地,根据证明中的另一部分，两个不同的最左推导产生不同的语法树。
- 最右推导类似。

二义性的来源

- 2+3*4
  - ① 2+(3*4)
  - ② (2+4)*4
- 操作符的优先级
- 2+3+4
  - ① (2+3)+4
  - ② 2+(3+4)
- 中缀操作符的结合性
- if e_1 then if e_2 then s_1 else s_2
  - ① if e_1 then {if e_2 then s_1 else s_2}
  - ② if e_1 then {if e_2 then s_1} else s_2

二义性是文法的性质

- 二义性是文法的性质不是语言的性质
- 对于平衡括号语言，有另一个CFG，这个没有二义性。

$$B \longrightarrow (RB \mid \varepsilon$$

$$R \longrightarrow) \mid (RR$$

  - B 文法初始符号，从它推导出平衡括号串。
  - R 生成右括号比左括号多一个的串。

例：把二义文法转换为无二义文法

- 二义文法：$B \longrightarrow BB \mid (B) \mid ()$
- 无二义文法：
  - $B \longrightarrow (RB \mid \varepsilon$
  - $R \longrightarrow) \mid (RR$
- 对于给定的平衡的括号的串通过从左到右扫描该串，构造出唯一最左推导。
- 如果我们需要扩展 B:
  - 若下一个符号是“(”使用 $B \longrightarrow (RB$扩展
  - 若输入串为空使用ε扩展
- 如果我们需要扩展 R:
  - 若下一个符号是“)”使用 $R \longrightarrow)$ 扩展
  - 若下一个符号是“(”使用 $R \longrightarrow (RR$ 扩展

分析过程

最左推导步骤:

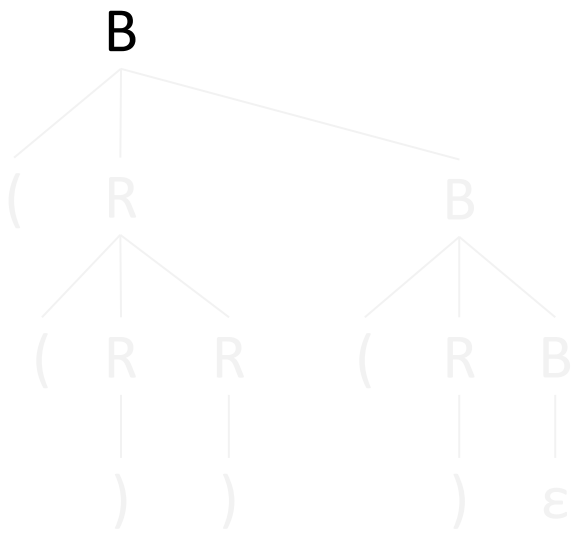
B

剩余输入串：

(( )) ( )

^

当前输入符号



$B \rightarrow (RB \mid \varepsilon \qquad R \rightarrow) \mid (RR$

The Parsing Process

剩余输入串：

(())()

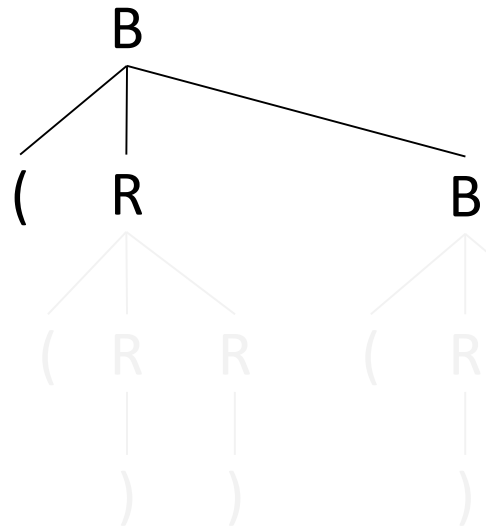
^

当前输入符号

最左推导步骤:

B

(RB



B⟶(RB | ε　　R⟶) | (RR

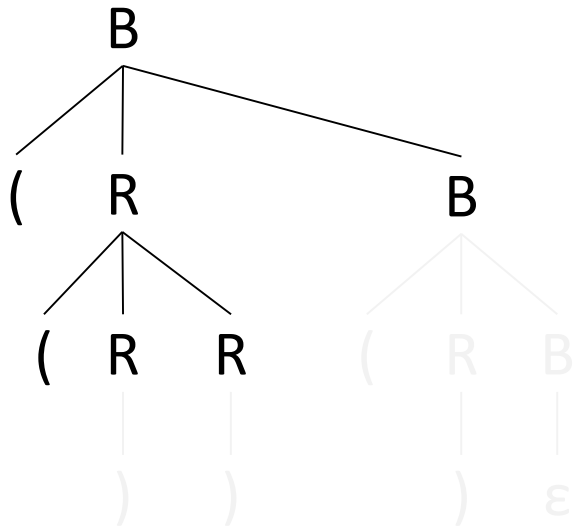
The Parsing Process

剩余输入串：

(()())

^

当前输入符号



最左推导步骤:

B

(RB

((RRB

B→(RB | ε　　R→) | (RR

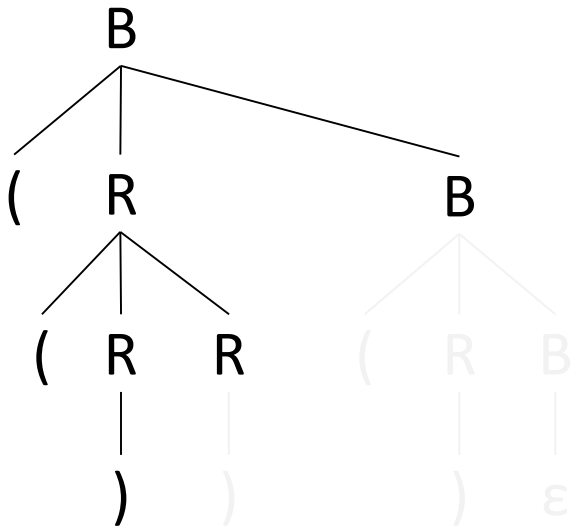
The Parsing Process

剩余输入串：

(( )) ( )

^

当前输入符号



最左推导步骤:

B

(RB

((RRB

(( ) RB

B→(RB | ε　　R→) | (RR

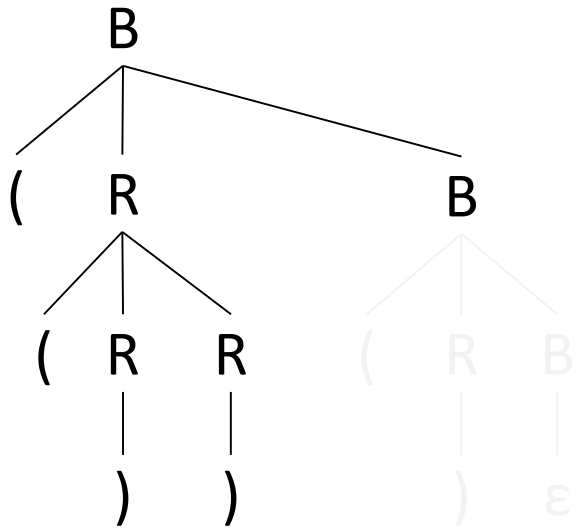
The Parsing Process

剩余输入串：

(()()

^

当前输入符号



最左推导步骤:

B

(RB

((RRB

(()RB

(())B

B→(RB | ε　　R→) | (RR

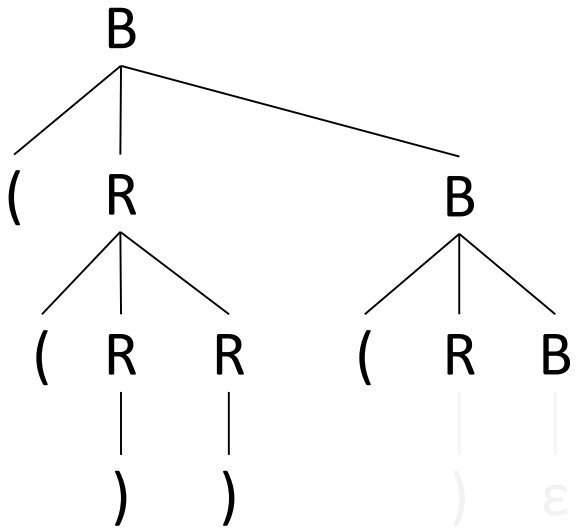
The Parsing Process

剩余输入串：

(())()

^

当前输入符号



最左推导步骤:

B

(RB

((RRB

(()RB

(())B

(())(RB

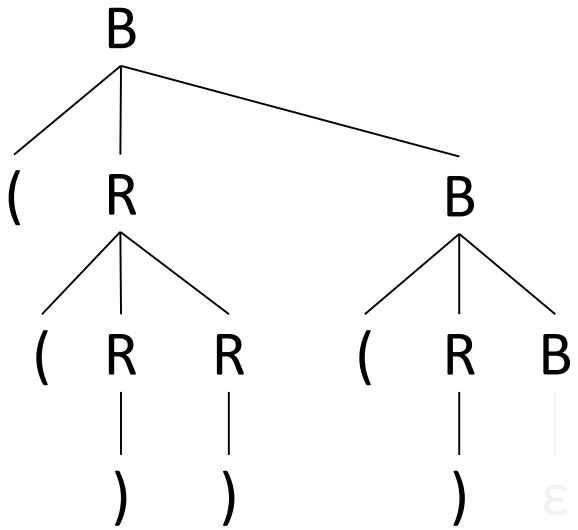
B→(RB | ε　　R→) | (RR

The Parsing Process

剩余输入串：

^

当前输入符号



最左推导步骤:

B

(RB

((RRB

(()RB

(())B

(())(RB

(())()B

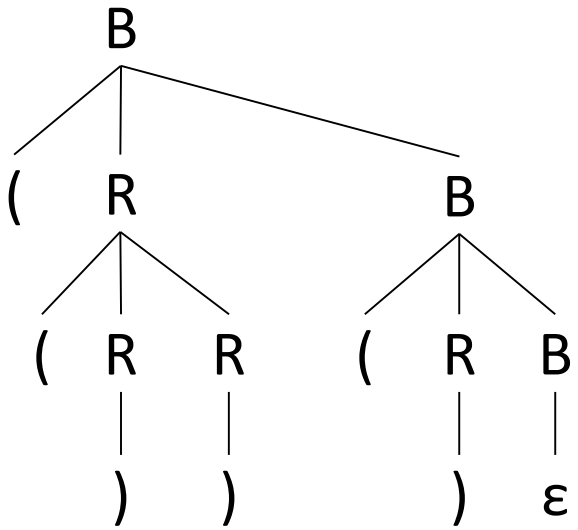
B→(RB | ε　　R→) | (RR

The Parsing Process

剩余输入串：

^

当前输入符号



最左推导步骤:

B

(RB

((RRB

(()RB

(())B

(()) (RB

(()) ()B

(()) ()

$B \rightarrow (RB \mid \varepsilon \qquad R \rightarrow) \mid (RR$

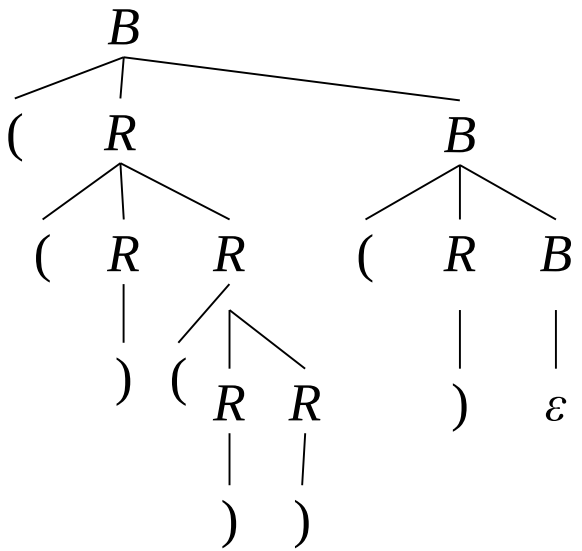
平衡括号文法的另一无二义文法

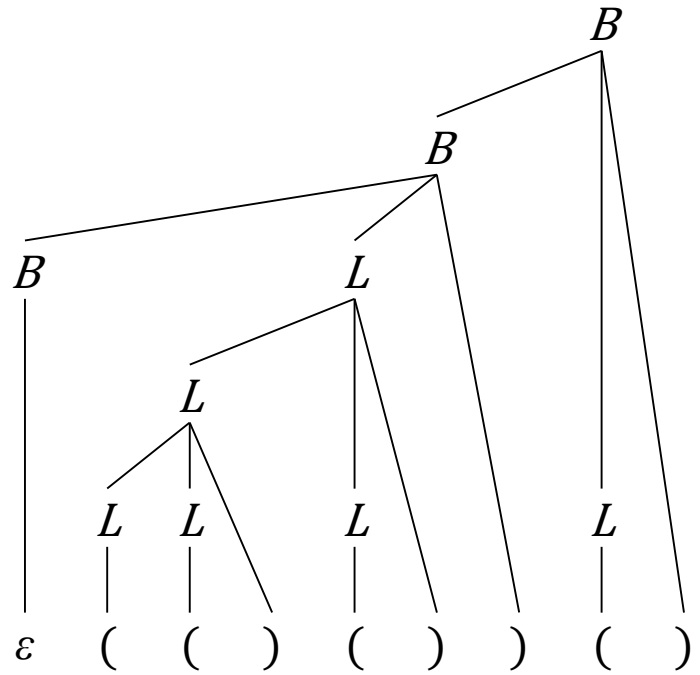
B⟶(RB | ε

R⟶) | (RR

B⟶BL) | ε

L⟶(| LL)





固有的二义性

- 消除二义性：通过分析产生二义性的原因，针对性地修剪文法得到无二义性文法。
- 分析定义CFL的多个文法：
  ① 都是无二义的
  ② 有一些是无二义的，其余都是二义的
  ③ 都是二义的
- 选择无二义的文法来进行CFL的语法分析，但不幸的是，有一些CFL是固有的二义的，即对它不存在无二义文法。
- 二义性文法是语法分析需要考虑的情形。

Example:固有的二义性

- 语言$\{0^i1^j2^k \mid i = j \text{ or } j = k\}$是固有的二义的。
- 直觉地，至少一些串形如$0^n1^n2^n$必定是由两个不同语法树产生的，一个基于对0和1的个数进行检测，而另一个基于对1和2的个数进行检测。

- -是左结合+是右结合的表达式2-3+4（同一优先级运算符的结合性不一致）

中缀运算符⊕的结合性

- ⊕是左结合的，如果$a \oplus b \oplus c=(a \oplus b) \oplus c$
- ⊕是右结合的，如果$a \oplus b \oplus c=a \oplus (b \oplus c)$
- ⊕是无结合性的，如果$a \oplus b \oplus c$则为非法
- 数学上，-和/是左结合的，+和*是结合的，左右都可以
- 计算上，因为满足结合律的运算符它的左、右结合性表现在精度、是否溢出等方面会有不同，所以左、右结合性只能二者选一。
- 一些语言如C选则左结合的+和*，这样就与数学上的-和/的左结合性保持一致，否则就会顶牛，对消除二义性没有帮助。
- 但是C中的赋值操作是右结合的，x=y=z被解释为x=(y=z)
- 另一些程序设计语言如SML有一些右结合操作。
- PASCAL语言的<和>都是无结合性的。即a<b<c出错。

消除文法二义性（ā修剪操作符）

CFG的等价性质：$\bar{a} \cdot G = G$

- $E \longrightarrow E \oplus E \mid d$
- $\oplus$是左结合的，
  - $E \longrightarrow E \oplus E' \mid E'$
  - $E' \longrightarrow d$
- $\oplus$是右结合的，
  - $E \longrightarrow E' \oplus E \mid E'$
  - $E' \longrightarrow d$
- 扩展到多个运算符时保持结合性一致
  - $E \longrightarrow E + E' \mid E - E' \mid E'$
  - $E' \longrightarrow d$

解决不同优先级操作符带来的二义性

- 每个优先级对应一个非终结符
- 如果一个表达式使用了某优先级的运算符，那么它的子表达式就不能使用较低优先级的运算符（除非出现在括号里）。
- 因此一个优先级的变元的候选式中不能出现那些对应于比它优先级低的变元（除非出现在括号里）。

- $E \longrightarrow E + T \mid T$
- $T \longrightarrow T * F \mid F$
- $F \longrightarrow (E) \mid d \mid i$

一种有二义性的文法

$S \longrightarrow AB \mid CD$

$A \longrightarrow 0A1 \mid 01$

$B \longrightarrow 2B \mid 2$

$C \longrightarrow 0C \mid 0$

$D \longrightarrow 1D2 \mid 12$

A 产生相等个数0和1

B 产生任意个数的 2

C 产生任意数目的 0

D 产生相等个数1和2

每个有相等数目的0, 1, 和2的串都有两个最左推导.如:

$S \Rightarrow AB \Rightarrow 01B \Rightarrow 012$

$S \Rightarrow CD \Rightarrow 0D \Rightarrow 012$

小结：文法、语言、程序

$P \longrightarrow \check{D}\ \check{S}$

$\check{D} \longrightarrow \varepsilon \mid \check{D}\ D\ ;$

$D \longrightarrow T\ d \mid T\ d(\check{A})\ \{\check{D}\ \check{S}\}$

$T \longrightarrow \text{int} \mid \text{void}$

$\check{A} \longrightarrow \varepsilon \mid \check{A}\ A\ ;$

$A \longrightarrow T\ d \mid T\ d()$

$\check{S} \longrightarrow S \mid \check{S}\ ;\ S$

$S \longrightarrow \text{if}\ (B)\ S \mid \text{if}\ (B)\ S\ \text{else}\ S \mid \text{while}\ (B)\ S \mid$
$\text{return}\ E \mid \text{print}\ d \mid \{\check{S}\} \mid d(\check{R})$

$B \longrightarrow B \wedge B \mid B \vee B \mid E\ r\ E \mid E$

$E \longrightarrow d = E \mid i \mid d \mid d(\check{R}) \mid E\ o\ E \mid (E)$

$\check{R} \longrightarrow \varepsilon \mid R$

$R \longrightarrow E \mid d()$

词法单元SCO，ID，LPA，RPA，LBR，RBR，INT，VOID，IF， ELSE，WHILE，RETURN，AND，OR，ROP={<, <=, >}，ASG，NUM，AOP={+, *, /}

```
int raw(int x;) {
        y = x+5;
        return y};
void foo(int y;) {
        int z;
        void bar(int x; int soo();) {
        if(x>3) bar(x/3, soo(),)
        else z = soo(x);
        print z};
    bar(y, raw(),)};
foo(6,)
```

$\mathcal{L}$ = {SCO，ID，LBK，RBK，LPA，RPA，LBR，RBR，INT，VOID，NUM，CMA，IF，ELSE，~~WHILE，~~RETURN，PRINT，ADD，~~MUL，~~DIV，~~AND，OR，~~ROP}

5.5 CFG化简

- 对文法进行哪方面的化简？

  5.5.1 去除无用符号

  5.5.2 去除ε产生式

  5.5.3 去除单位产生式

CFG的几种等价性质

对于CFG G有

- $a\cdot G$的文法符号都是可达的且$L(a\cdot G)=L(G)$
- $g\cdot G$的变元都是产出的且$L(g\cdot G)=L(G)$
- $\bar{e}\cdot G$不含ε产生式且$L(\bar{e}\cdot G)=L(G)$
- $\bar{u}\cdot G$ 不含单位产生式且$L(\bar{u}\cdot G)=L(G)$

5.5.1 有用与无用的文法符号

- 出现在一个句子的成功推导中的文法符号都是有用符号
  - 每个句子的始于初始符号的推导都是它的一个成功推导
- 对CFG (V, T, P, S)，若有$S \Rightarrow^* \alpha X\beta \Rightarrow^* w$，$\alpha, \beta \in (V \cup T)^*$，$w \in T^*$ 那么X是有用的文法符号（有用符号）
- 分为两个方面来看：
  - ① $S \Rightarrow^* \alpha X\beta$　　X是可达符号；X是**可达的**
  - ② $X \Rightarrow^* w$　　X是有产出的符号；X是**产出的**

结论：

1. 由文法初始符号推导出的任意串中的文法符号都是可达的。
2. 能推导出终结符串的文法符号都是产出的。
3. 既是可达的又是产出的文法符号是有用的。
4. 一个文法的文法符号不是有用的便是无用的。

例：有用、无用的文法符号

- 终结符总是产出的，但不一定是可达的；
- 对变元而言，各种情形都有可能。

  ① $S \Rightarrow^* \alpha X \beta$　　X是可达的

  ② $X \Rightarrow^* w$　　X是产出的

- 例：$S \rightarrow AB \quad A \rightarrow aA \mid a \quad B \rightarrow AB$
- 可达符号：S，A，B，a
- 不可达符号：无
- 有产出的符号：A，a
- 无产出的符号：S，B
- 因为文法初始符号S是无产出的，所以该语言是空（不考虑）

去除无用符号

- 对于CFG $G=(V,T,P,S)$，$L(G)\neq\emptyset$
- 从G中去除无产出变元得到$gG=(V_g,T,P_g,S)$，其中：

  V_g是V中去掉无产出成员所得；

  P_g是P中去掉含有无产出变元的成员（即产生式）。
- 从G中去除不可达符号得到CFG $aG=(V_a,T_a,P_a,S)$，其中：

  T_a是从T中去掉不可达成员；V_a是V中去掉不可达成员；

  P_a是P去掉含有不可达符号的成员。

- 例G：$S\longrightarrow AB|a \quad A\longrightarrow b$
  - V_g=？；P_g=？
- 则gG：$S\longrightarrow a \quad A\longrightarrow b$
- 那么agG：$S\longrightarrow a$
- 对于G'：$S\longrightarrow AB|B \quad A\longrightarrow b$会怎样？$V_g$中不能没有$S$！

例：去除无用符号

- 从文法中先去除无产出变元，进而去除不可达符号。

- 例G：$S\longrightarrow AB|C \quad A\longrightarrow aA|a \quad B\longrightarrow bB \quad C\longrightarrow c$
  - $V=\{S, A, B, C\}$；$T=\{a, b, c\}$；P；S

- gG：$S\longrightarrow C \quad A\longrightarrow aA|a \quad C\longrightarrow c$
  - $V_g=\{S, A, C\}$；$T=\{a, c\}$；P_g；S

- agG：$S\longrightarrow C \quad C\longrightarrow c$
  - $V_{ga}=\{S, C\}$；$T_a=\{c\}$；P_a；S

- 思考：从G中去除不可达符号，进而去除无产出变元，如何？

定理证明

- **定理**5.6　对于CFG $G=(V, T, P, S)$，$L(G)\neq\emptyset$，令$g\cdot G=(V_g, T, P_g, S)$，$ag\cdot G=(V_{ga}, T_a, P_{ga}, S)$，则$ag\cdot G$中没有无用符号，且$L(ag\cdot G)=L(G)$。

证明：(1) $ag\cdot G$中没有无用符号。

- 对于每一个$X\in T\cup V_g$，则X也属于$T\cup V$，如果$X\Rightarrow^*_G w, w\in T^*$，那么这个推导中所有符号都是产出的，所以$X\Rightarrow^*_{gG} w$。
- 对于每一个$X\in T\cup V_g$，如果$S\Rightarrow^*_{gG}\alpha X\beta, \alpha, \beta\in(V_g\cup T)^*$，那么这个推导中所有符号都是可达的，所以有$S\Rightarrow^*_{agG}\alpha X\beta$，并且$X\in V_{ga}\cup T_{ga}, \alpha, \beta\in(V_{ga}\cup T_{ga})^*$。

证明：(2) 证明等价性质$L(ag\cdot G)=L(G)$。

- 由于$T_a\subseteq T$，$V_{ga}\subseteq V$，并且$P_{ga}\subseteq P$，所以$L(ag\cdot G)\subseteq L(G)$。
- 又由于$S\Rightarrow^*_G w, w\in T^*$中所有出现的符号都是有用的，所以有$S$

发现有产出变元的算法

- 若$X \Rightarrow^* w$，则X是产出的。

- 根据CFG的产生式判断变元A是否是产出的：
  - 基础：若存在产生式$A \longrightarrow w$，其中w不含变元，那么A是产出的。
  - 归纳：若存在产生式$A \longrightarrow \alpha$，其中α仅由终结符和有产出变元组成，那么A是产出的。

- 证明：根据推导$X \Rightarrow^* w$的步数归纳，0步成立；假定n-1步成立；那么n步时，$X \Rightarrow \alpha \Rightarrow^* w$中第一步对应产生式$A \longrightarrow \alpha$，且$\alpha$中每个符号都推导出$w$的一个子串，并且推导步数小于$n$，那么这些符号都是产出的，因此算法能发现$A$是产出的。

例：发现有产出变元

- $S \longrightarrow AB|C \quad A \longrightarrow aA|a \quad B \longrightarrow bB \quad C \longrightarrow c$

- 基础：A 和 C 可以被确定有产出，因为 $A \longrightarrow a$ 和 $C \longrightarrow c$。
- 归纳：S 可被确定有产出，因为 $S \longrightarrow C$。
- 没有别的可以被发现的了。

发现可达符号的算法

- 若$S \Rightarrow^* \alpha X \beta$，则$X$是可达的。

- 根据CFG的产生式判断某变元A是否是可达的：
  - 基础：文法初始符号S可达。
  - 归纳：如果A可达，并且存在产生式$A \longrightarrow \alpha$，那么所有α中的符号均可达。

- 证明：对从S开始推导的步数归纳。0步成立；设小于n步成立；那么n步时如果有$S \Rightarrow^* \alpha A \beta \Rightarrow \alpha \gamma \beta$，其中$A \longrightarrow \gamma$。除最后一步直接推导，前面的步数小于$n$，所以$A$是可以被发现的，那么$\gamma$中的符号也能被发现。

例：发现可达符号

- 例：$S \rightarrow AB \quad A \rightarrow C|\varepsilon \quad C \rightarrow c \quad B \rightarrow bB \quad D \rightarrow \varepsilon$
- 基础：S被发现，因为文法初始符号是可达的。
- 归纳：A和B被发现，因为有产生式$S \rightarrow AB$，即有$S \Rightarrow^* AB$；
- 接着C被发现，因为$A \rightarrow C$；
- b被发现因为$B \rightarrow bB$；
- 接着c被发现因为$C \rightarrow c$；
- 再没有别的可发现的了。

- 思考：ε是不是可达的？

5.5.2 去除ε产生式

- ε产生式$A \longrightarrow \varepsilon$是有用的但不是必须的。如果语言中不考虑空串的话我们定义它时就不需要ε产生式了，或者含有ε产生式的CFG可以等价转化为无ε产生式的CFG。
- 发现可空变元。
- 可空变元在候选式中不出现的话，事实上就反映了它推导ε这种情况，如此就已经起到了ε产生式的作用。
- 去除ε产生式。

- 定义：CFG (V,T,P,S)，$A \in V$是可空变元，当且仅当$A \Rightarrow^{*} \varepsilon$。

- 例：$S \longrightarrow AcB \quad A \longrightarrow aA|\varepsilon \quad B \longrightarrow bB|A$
- 可空变元：A；B

发现可空变元的算法

- CFG $G=(V, T, P, S)$的$A \in V$是可空变元，当且仅当$A \Rightarrow^* \varepsilon$。

- 基础：若$A \longrightarrow \varepsilon$是$G$的产生式，那么$A$可空。
- 归纳：若G有产生式$A \longrightarrow C_1 \ldots C_k$, 其中对于$i=1,2,\ldots,k$, 有$C_i \in V$且是可空的，那么$A$是可空的。

- 例：$S \longrightarrow AB \quad A \longrightarrow aA|\varepsilon \quad B \longrightarrow bB|A$
- 基础：A可空因为$A \longrightarrow \varepsilon$。
- 归纳：B可空因为$B \longrightarrow A$；再者，S可空因为$S \longrightarrow AB$。

定理证明

定理5.7 在任何文法G中，上面的算法生成的恰好是全部的可空符号。

- 变元A被算法找出当且仅当$A \Rightarrow^* \varepsilon$

- 证明：根据推导$A \Rightarrow^* \varepsilon$的步数归纳，第1步成立即算法发现$A$可空；假定小于$n$步时成立；那么$n$步时，不失一般性，有$A \Rightarrow C_1 C_2 \cdots C_k \Rightarrow^* \varepsilon$，其中对于$i=1, 2, \cdots, k$，$C_i$小于$n$步推导出$\varepsilon$，根据归纳假设$C_i$可空，因此$A$可空。

- 若算法发现A可空，那么$A \Rightarrow^* \varepsilon$，证明对应算法中归纳过程（留作练习）。

去除ε产生式

- 变形产生式：若产生式$A\longrightarrow\alpha$ 的候选式α中有m个符号是可空变元，$m\geqslant 0$，那么这m个变元的每一个出现和不出现，就形成了α的2^m个变形版本。其中α又称为自己的原形版本。
- $m=0$时？只有原形版本
- $m=|\alpha|$时？ 有个$A\longrightarrow\varepsilon$版本

性质：如果使用变形候选式推导出w，那么，使用原形候选式亦推导出w。

- 举例：若A 可空，那么产生式 $B\longrightarrow CAD$ 的变形版本为：
- $B\longrightarrow CAD$；$B\longrightarrow CD$
- 举例：若A和B可空，那么产生式$S\longrightarrow AaBDbAbC$的变形版本：
- $S\longrightarrow aDbbC|\ aDbAbC|\ aBDbbC|\ aBDbAbC|\ AaDbbC|\ AaDbAbC|$

算法：去除ε产生式

输入：$G=(V,T,P,S)$

输出：$\bar{e}G=(V,T,P_{\bar{e}},S)$

- $P_{\bar{e}}$初始化为空集合；
- 找出G的所有可空变元；
- 对P中每个非ε产生式，将它的变形全部加入$P_{\bar{e}}$；
- $P_{\bar{e}}$中去掉以ε为候选式的所有产生式。

例：去除ε产生式

G：$S \to ABC$ $A \to aA \mid \varepsilon$ $B \to bB \mid \varepsilon$ $C \to \varepsilon$

- 其中A、B、C和S 都是可空的。

ēG：

$S \to ABC \mid AB \mid AC \mid A \mid BC \mid B \mid C$

$A \to aA \mid a$

$B \to bB \mid b$

例：去除ε产生式

G'：$S \longrightarrow AB$ $A \longrightarrow aAA \mid \varepsilon$ $B \longrightarrow bBB \mid \varepsilon$

- 其中A和B都是可空的。

$\bar{e}G'$：

$S \longrightarrow AB \mid A \mid B$

$A \longrightarrow aAA \mid aA \mid a$

$B \longrightarrow bBB \mid bB \mid b$

- $L(G') = L(\bar{e}G') \cup \{\varepsilon\}$

定理5.8

- 如果文法$\bar{e}G$是利用去除ε产生式的算法从G得出的，则$L(\bar{e}G) = L(G)\backslash\{\varepsilon\}$。
- 令$G=(V, T, P, S)$
- 令$\bar{e}G = (V, T, P_{\bar{e}}, S)$

- 证明对任意变元$A\in V$，$A \Rightarrow^*_{\bar{e}G} w$当且仅当$A \Rightarrow^*_{G} w$且$w\neq\varepsilon$。

- 对推导步数进行归纳。

定理证明

- （当）若$w \neq \varepsilon$且$A \Rightarrow^*_G w$，则$A \Rightarrow^*_{\bar{e}G} w$。

- 基础：第1步时，$(A, w) \in P$，由于$w \neq \varepsilon$，所以$(A, w) \in P_{\bar{e}}$。

- 归纳：假设步数小于n时结论成立，那么n步时，不失一般性有$A \Rightarrow_G X_1 X_2 \cdots X_k \Rightarrow^*_G w$。$w$可被分段成$w = w_1 w_2 \cdots w_k$，对于$i = 1, 2, \cdots, k$，有$X_i \Rightarrow^*_G w_i$，并且其步数小于$n$。那么根据归纳假设，若$w_i \neq \varepsilon$，则有$X_i \Rightarrow^*_{\bar{e}G} w_i$。
- 我们把推导为$w_j = \varepsilon$的X_j去掉，对于所有j，结果记为α。那么α是$X_1 X_2 \cdots X_k$的一个变形，则有$\alpha \Rightarrow^*_{\bar{e}G} w$，同时$A \longrightarrow X_1 X_2 \cdots X_k$和$A \longrightarrow \alpha$都在$\bar{e} \cdot G$中，因此，$A \Rightarrow_{\bar{e}G} \alpha \Rightarrow^*_{\bar{e}G} w$。

定理证明

- （仅当）若$A \Rightarrow^*_{\bar{e}G} w$则$A \Rightarrow^*_G w$且$w \neq \varepsilon$。
- 基础：推导步数为1时，必有$(A,w) \in P_a$，这是原形产生式所以$(A,w) \in P$而且$w \neq \varepsilon$。
- 归纳：假设推导步数小于n时结论成立，那么推导步数为n时，
- $A \Rightarrow_{\bar{e}G} \alpha \Rightarrow^*_{\bar{e}G} w$，由于$\alpha \Rightarrow^*_{\bar{e}G} w$的步数小于$n$，所以根据归纳假设$\alpha \Rightarrow^*_G w$。
- 我们找出$(A, \alpha) \in P_{\bar{e}}$的原形产生式$(A, \delta) \in P_{\bar{e}}$，就有$(A, \delta) \in P$并且$A \Rightarrow_G \delta \Rightarrow^*_G w$。

5.5.3 去除单位产生式

- 单位产生式：右部为单个变元的产生式。

- 思路：若$A \Rightarrow^* B$是通过一系列单位产生式完成，并且$B \longrightarrow \alpha$是一个非单位产生式，那么给文法中增加产生式$A \longrightarrow \alpha$。
- 此项工作完成后，删掉所有单位产生式。

- $S \longrightarrow A|a$
- $A \longrightarrow B|b|Cc$
- $B \longrightarrow d|e$

- $S \longrightarrow d|e|b|Cc|a$
- $A \longrightarrow d|e|b|Cc$

发现单位产生式的算法

- 关键点：找出所有单位对(A, B)使得$A \Rightarrow^* B$仅通过一系列单位产生式得到。
- 基础： 对于任何变元，(A, A)是单位对。
- 归纳： 若已知(A, B)是单位对且$B \longrightarrow C$是一个单位产生式，那么(A, C)是单位对。

- 例：α，β，γ，δ，η都不是单个变元，
- $A \longrightarrow B \mid \alpha$
- $B \longrightarrow C \mid \beta$
- $C \longrightarrow D \mid \gamma$
- $D \longrightarrow \delta \mid \eta$
- 单位对：(A, B)，(A, C)，(A, D)，(B, C)，(B, D)，(C, D)

证明算法正确工作

- 基础： 对于任何变元，(A, A)是单位对。
- 归纳： 若已知(A, B)是单位对且$B \longrightarrow C$ 是一个单位产生式，那么(A, C)是单位对。

- 证明算法能发现给定文法的所有单位对。
- 按照找出单位对 (A, B)的次序归纳，能表明$A \Rightarrow^* B$ 是通过单位产生式得到。
- 相反地，对经过单位产生式得到的$A \Rightarrow^* B$ 的推导步数进行归纳，能够表明 单位对(A, B) 被发现。

去除单位产生式的算法

- 输入：CFG $G=(V,T,P,S)$
- 输出：$\bar{u}\,G=$ CFG $(V,T,P_{\bar{u}},S)$

求出G的所有单位对；

对每个单位对(A,B)，把所有的$(A,\gamma)\in P$加入$P_{\bar{u}}$，其中γ不是单个变元且$(B,\gamma)\in P$。

证明：对任意T上的符号串w有$w\in L(G)$，当且仅当$w\in L(\bar{u}\,G)$

- （当）$S\Rightarrow^*_{\bar{u}G} w$为1步时有$(S,w)\in P$或者存在单位对$(S,A)$且$(A,w)\in P$。假设步数小于$n$时结论成立，那么第$n$步时，$S\Rightarrow_{\bar{u}G} X_1X_2\cdots X_k\Rightarrow^*_{\bar{u}G} w$，其中对于$i=1,2,\cdots k$，有$X_i\Rightarrow^*_{\bar{u}G} w_i$，那么根据归纳假设有$X_i\Rightarrow^*_{G} w_i$。因此，对于$(S,X_1X_2\cdots X_k)\in P_{\bar{u}}$，或者有$(S,X_1X_2\cdots X_k)\in P$，或者存在单位对$(S,A)$且$(A,X_1X_2\cdots X_k)\in P$
- （仅当）仿照上面思考。

例：去除单位产生式

1. 求出G的所有单位对；
2. 对每个单位对(A, B)，把所有的$(A, \delta) \in P$加入$P_{\bar{u}}$，其中δ不是单个变元且$(B, \delta) \in P$。

- 例：α，β，γ，δ，η都不是单个变元，

$A \longrightarrow B|\alpha \;\; B \longrightarrow C|\beta \;\; C \longrightarrow D|\gamma \;\; D \longrightarrow \delta|\eta$

单位对：(A, B)，(A, C)，(A, D)，(B, C)，(B, D)，(C, D)

从而得到$\bar{u}G$：

$A \longrightarrow \delta \mid \eta \mid \gamma \mid \beta \mid \alpha$

$B \longrightarrow \delta \mid \eta \mid \gamma \mid \beta$

$C \longrightarrow \delta \mid \eta \mid \gamma$

$D \longrightarrow \delta \mid \eta$

例：去除单位产生式

- 例：G_{ETF}：

  $E \longrightarrow E+T \mid T$

  $T \longrightarrow T*F \mid F$

  $F \longrightarrow (E) \mid d$

- 单位对：(E, T)，(E, F)，(T, F)。
- $\bar{\mathrm{u}}G_{\mathrm{ETF}}$：

  $E \longrightarrow E+T \mid T*F \mid (E) \mid d$

  $T \longrightarrow T*F \mid (E) \mid d$

  $F \longrightarrow (E) \mid d$

清理文法

- 定理5.10 若L是一个CFL，那么有一个$L\backslash\{\varepsilon\}$的CFG有如下特征：
  - 不含无用符号。
  - 不含ε产生式。
  - 不含单位产生式。
- 每个文法符号都是有用的；每个候选式或者是单个终结符或者是长度大于1的文法符号串。

证明：由于L是一个CFL，那么必存在语言识别器，设为CFG G。那么$ag\bar{u}\bar{e}G$就是具备定理描述特征的文法。理由陈述为：$\bar{e}\cdot G$的每个候选式长度大于等于1；$\bar{u}\bar{e}\cdot G$的每个候选式或为终结符或长度大于1，因为使用算法5.5对$\bar{e}\cdot G$去除单位产生式不会引入ε产生式。最后对文法$\bar{u}\bar{e}\cdot G$去除无用符号不会增加产生式，所以结果文法$ag\bar{u}\bar{e}\cdot G$具备定理所述特征。

清理文法

由$ag\bar{u}\bar{e}\cdot G$得到清理文法G的步骤：

- 首先消除ε产生式
  - 对G做$\bar{e}$修剪操作得到$\bar{e}G$
- 其次消除单位产生式
  - 对$\bar{e}G$做$\bar{u}$修剪操作得到$\bar{u}\bar{e}G$
- 接着消除无产出变元
  - 对$\bar{u}\bar{e}G$做g修剪操作得到$g\bar{u}\bar{e}G$
- 最后消除不可达变元
  - 对$g\bar{u}\bar{e}G$做a修剪操作得到$ag\bar{u}\bar{e}G$

小结

- CFL语言识别器CFG，代数定义，变元与终结符、列表表示，
- 判定性质：推导和归约、直接推导、最左和最右推导、语言、句子、句型、语法树、产物、
- 推导与语法树扩展对应，归约与语法树构建对应
- 二义性与消除文法二义性（运算符优先级原因）
- 等价性质：四种修剪操作
- 产生式左部与右部、候选式，符号串替换

- 习题5.1(1)、5.4、5.6~8、5.10、5.13